

معرفی پروژه :

این پروژه با هدف پیاده‌سازی یک دیکدر کد مورس طراحی شده است. در این پروژه، سیگنال‌های مورس از طریق یک دکمه ورودی دریافت، زمان‌بندی و به الگوی متناظر (نقطه و خط) تبدیل می‌شوند. سپس، این الگوها دیکد میشوند.

اهمیت پروژه در صنعت:

این پروژه کاربردهای صنعتی و آموزشی متنوعی دارد؛ از شناسایی خودکار کد مورس در سامانه‌های ناوبری هوایی و دریایی، تا استفاده در فناوری‌های کمکی برای افراد دارای معلولیت. همچنین در ارتباطات نظامی و اضطراری در شرایط نویزی کاربرد دارد و می‌تواند به‌عنوان یک ماژول آموزشی در بردهای FPGA برای آموزش پردازش سیگنال دیجیتال مورد استفاده قرار گیرد.

شرح پروژه :

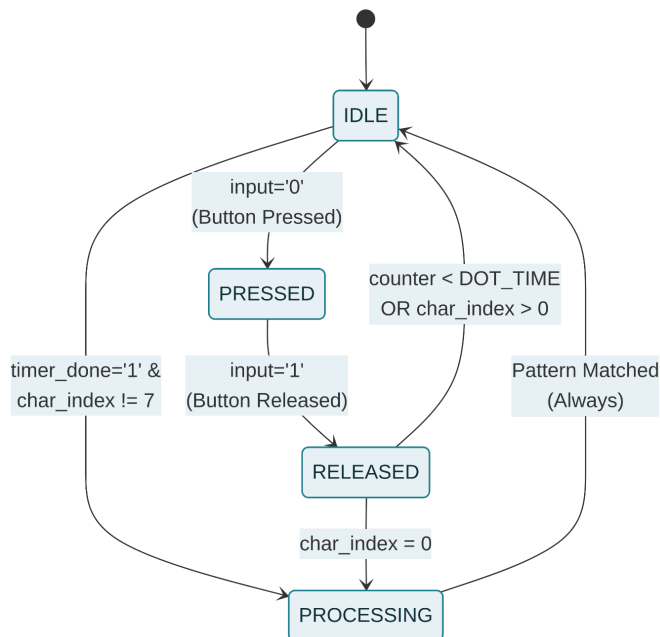
```
entity morse is
  port(
    clk      : in std_logic;
    input     : in std_logic;
    outleddot : out std_logic;
    outleddas : out std_logic;
    outledsep : out std_logic;
    state_iden1: out std_logic;
    state_iden2: out std_logic;
    output     : out std_logic_vector(7 downto 0);
    counter_o  : out integer
  );
end entity;
```

ENTITY morse بلاک اصلی طراحی است و پورت‌های ورودی و خروجی ماژول را تعریف می‌کند. این ماژول شامل ورودی کلاک، ورودی دکمه برای وارد کردن کد مورس (تشخیص نقطه و خط بر اساس زمان فشرده شدن)، خروجی‌های LED برای نمایش نقطه، خط تیره و وضعیت فعال بودن دکمه، خروجی برای نشان دادن state فعلی، و یک خروجی ۸ بیتی برای نمایش کاراکتر دیکدشده‌ی مورس است.

```
constant DOT_TIME    : integer := 20; -- ~200 ns @ 100MHz
constant DASH_TIME   : integer := 40; -- ~400 ns
constant IDLE_TIME    : integer := 80; -- ~800 ns
```

در این بخش، تعداد سیکل های کلاک لازم برای تشخیص نقطه، خط و پایان ورودی تعیین شده است. با فرکانس ۱۰۰ مگاهرتز، زمان تشخیص نقطه بیش از ۲۰۰ نانوثانیه، خط بیش از ۴۰۰ نانوثانیه و ارسال ورودی برای دیکد بیش از ۸۰۰ نانوثانیه در نظر گرفته شده است.

تعریف وضعیت های FSM :



طبق FSM، سیستم برای دریافت و ترجمه کد مورس چهار وضعیت اصلی دارد: **IDLE** برای انتظار ورودی و تشخیص پایان کاراکتر، **PRESSED** برای اندازه گیری مدت فشار کلید و تعیین نقطه یا خط، **RELEASED** برای ثبت نقطه یا خط در حافظه و بازگشت به حالت انتظار، و **PROCESSING** برای مقایسه الگوی ذخیره شده با جدول مورس، تولید خروجی و بازنشانی سیستم.

سیگنال های دیکدر کد مورس :

```

signal char_index      : integer range 0 to 7 := 7;
signal last_index      : integer range 0 to 7 := 7;
signal char_pattern    : std_logic_vector(7 downto 0) := (others => '0');
signal decoded_char    : std_logic_vector(7 downto 0) := "00000000";
signal counter         : integer := 0;
signal done_counter    : integer := 0;
signal timer_done      : std_logic := '0';
  
```

char_index موقعیت بیت فعلی، **char_pattern** الگوی ۸ بیتی مورس، و **decoded_char** نتیجه دیکد شده را نگه می دارند. مدت فشار دکمه را اندازه می گیرد و **done_counter** زمان بیکاری سیستم را برای تشخیص پایان کاراکتر می سنجد؛ **timer_done** با رسیدن به زمان مشخص فعال می شود.

جدول جست و جوی کد مورس :

```

TYPE morse_rom IS ARRAY (0 TO 35) OF STD_LOGIC_VECTOR(7 DOWNTO 0);
CONSTANT morse_lut : morse_rom := (
    "01-----", "1000-----", "1010-----", "100-----", "0-----", "0010-----",
    "110-----", "0000-----", "00-----", "0111-----", "101-----", "0100-----",
    "11-----", "10-----", "111-----", "0110-----", "1101-----", "010-----",
    "000-----", "1-----", "001-----", "0001-----", "011-----", "1001-----",
    "1011-----", "1100-----", "11111-----", "01111-----", "00111-----", "00011-----",
    "00001-----", "00000-----", "10000-----", "11000-----", "11100-----", "11110-----"
);

```

در این بخش، آرایه **morse_rom** شامل ۳۶ الگوی ۸ بیتی کد مورس تعریف شده و **morse_lut** با الگوهای حروف A-Z و اعداد 0-9 مقداردهی می شود؛ به طوری که ایندکس های ۰-۲۵ برای حروف و ۲۶-۳۵ برای اعداد هستند و بیت های «-» به صورت *don't care* استفاده می شوند.

شمارنده Idle Time :

```

process(clk)
begin
    if rising_edge(clk) then
        if done_counter = IDLE_TIME then
            timer_done <= '1';
        elsif state = IDLE then
            done_counter <= done_counter + 1;
            timer_done <= '0';
        end if;
        if state /= IDLE or char_index = 7 then
            done_counter <= 0;
        end if;
    end if;
end process;

```

این پراسس زمان بیکاری سیستم را در حالت **IDLE** می شمارد و با رسیدن به **IDLE_TIME** سیگنال **timer_done** را فعال می کند. اگر ورودی جدید آغاز شود یا **char_index = 7** باشد، شمارنده ریست می شود.

قسمت اصلی FSM ، گرفتن کاراکتر ورودی و دیکد آن

در این بخش، به طور گام به گام و بر اساس کد ارائه شده، توضیح می دهیم که چگونه سیستم کد مورس را از ورودی دکمه دریافت و در نهایت دیکد میکند.

1. گرفتن ورودی و تشخیص پایان کاراکتر

```
when IDLE =>
  if input = '0' then -- button pressed
    state <= PRESSED;
    counter <= 0;
  elsif timer_done = '1' and char_index /= 7 then
    state <= PROCESSING;
  end if;
```

در حالت IDLE، سیستم منتظر فشردن دکمه برای شروع کاراکتر (انتقال به PRESSED و صفر شدن counter) یا پایان کاراکتر (رها شدن دکمه به مدت IDLE_TIME، فعال شدن timer_done و $\text{char_index} \neq 7$) است تا وارد PROCESSING شود و کاراکتر دیکد گردد.

2. اندازه گیری مدت زمان فشار دکمه

```
when PRESSED =>
  counter <= counter + 1;
  if input = '1' then -- button released
    state <= RELEASED;
  end if;
```

در وضعیت PRESSED، سیستم مدت فشار دکمه را با counter اندازه می گیرد و تا رها شدن دکمه در این وضعیت باقی می ماند، سپس به وضعیت RELEASED می رود.

3. تشخیص نقطه یا خط بودن کاراکتر

```
when RELEASED =>
  if counter >= DASH_TIME then
    char_pattern(char_index) <= '1';
    outleddas <= '1';
    outleddot <= '0';
    char_index <= char_index - 1;
  elsif counter >= DOT_TIME then
    char_pattern(char_index) <= '0';
    outleddot <= '1';
```

```

        outleddas <= '0';
        char_index <= char_index - 1;
    else
        state <= IDLE;
    end if;

    if char_index = 0 then
        state <= PROCESSING;
    else
        state <= IDLE;
        counter <= 0;
    end if;

```

در وضعیت **RELEASED**، سیستم با **counter** نوع ورودی را تشخیص داده و در **char_pattern** ذخیره می‌کند ('1' برای خط، '0' برای نقطه) و فشارهای کوتاه نادیده گرفته می‌شوند. پس از ثبت هر بیت، **char_index** کاهش می‌یابد؛ اگر به صفر برسد، سیستم به **PROCESSING** می‌رود، در غیر این صورت به **IDLE** بازمی‌گردد و **counter** صفر می‌شود.

4. دیکد و نمایش کاراکتر

```

when PROCESSING =>
    decoded_char <= "11111111";
    -- Match pattern
    for i in 0 to 35 loop
        if char_pattern(7 downto char_index + 1) & complement_element(char_index downto 0) = morse_lut(i) then
            decoded_char <= std_logic_vector(to_unsigned(i, 8)); -- + to_unsigned(65,8)
            exit;
        end if;
    end loop;
    output <= decoded_char;

    char_index <= 7;
    char_pattern <= (others => '0');
    state <= IDLE;

```

در وضعیت **PROCESSING**، سیستم الگوی نقطه و خط ذخیره‌شده در **char_pattern** را به یک کاراکتر تبدیل می‌کند. ابتدا **decoded_char** با مقدار خطای پیش‌فرض تنظیم می‌شود، سپس الگو بر اساس **char_index** محاسبه و الگو با جدول **morse_lut** مقایسه می‌گردد تا ایندکس کاراکتر متناظر در **decoded_char** ذخیره شود. در پایان، **char_index** و **char_pattern** بازنشانی شده و سیستم به حالت **IDLE** بازمی‌گردد تا برای دریافت کاراکتر بعدی آماده باشد.